Trabalho 3 — Produto Evoluído + Refatoração

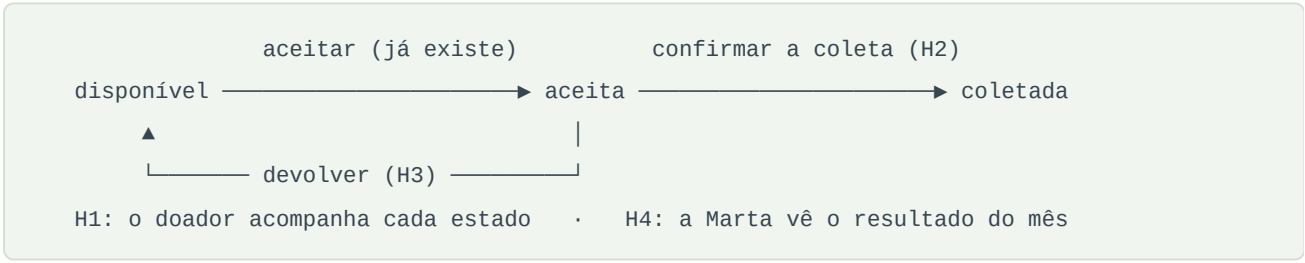
- **Unidade:** 3 — Construção · **Valor:** 3,0 (2,0 do artefato do grupo + 1,0 da defesa individual)
- **Formato:** produção em grupo (3 a 5 pessoas), **entrega individual** na atividade do Trabalho 3 no Teams
- **Prazo: 26/11** (dia da Prova 3), no horário da atividade no Teams
- **Nível de IA:** colaboradora

Este documento é o roteiro completo do trabalho. Ele é **iterativo**: o produto cresce em **quatro histórias**, uma por iteração, e cada iteração passa pelo **mesmo ciclo** — cortar, testar, implementar, revisar, integrar, mostrar e validar. Ao fim de cada iteração há um **"Pronto quando"**: só comece a próxima quando ele estiver verdadeiro.

Visão geral

O problema. Hoje o Prato Cheio faz uma coisa: o doador publica, a ONG aceita e a doação some da lista. Depois do aceite, ninguém sabe o que acontece. O doador não sabe se alguém vai buscar; a ONG que desiste não tem como devolver; ninguém registra a coleta; e a Marta não consegue mostrar o resultado que prometeu na análise — **reduzir o tempo entre "comida disponível" e "comida coletada"**.

O que o grupo constrói. Quatro histórias que completam a vida de uma doação. Cada uma depende da anterior, e juntas elas fecham o ciclo que a análise da Unidade 1 começou:



Iteração	História	Quem ganha	Até
0	**Base:** produto sobre PostgreSQL, CI verde, definição de pronto, os estados da doação decididos	o grupo	15/11
1	**H1 · O doador acompanha as suas doações**	doador	17/11
2	**H2 · O voluntário confirma a coleta**	voluntário e ONG	19/11
3	**H3 · A ONG devolve uma doação aceita** — e a Refatoração 2	ONG	22/11
4	**H4 · A Marta vê o resultado do mês**	Marta	24/11
F	**Fechamento:** validação, demo, documento, retrospectiva, `entrega-3`	todos	25/11

Muito já está feito. As Aulas 10 a 12 deixaram a definição de pronto, o fluxo de PR com testes e a migração para PostgreSQL. Este trabalho usa tudo isso — e acrescenta o que a construção pede: histórias de verdade, uma refatoração que nasce da necessidade e a validação contra o que foi combinado.

Regras que valem para tudo:

- Toda mudança entra por **PR revisado** por outro integrante, com **CI verde** e a **definição de pronto** cumprida. Nada de commit direto na `main`.
- **Um PR por vez** na `main`: branch atualizada antes do merge, **CI da `main` conferido** depois. `main` vermelha é prioridade do grupo — ninguém integra nada até ela voltar.
- **Cada história tem um dono e um revisor diferentes.** Em grupo de 3, alguém é dono de duas; em grupo de 5, a quinta pessoa é dona do fechamento.
- Os nomes de arquivos e seções são **exatamente** os pedidos aqui: a correção procura por eles.
- **Cada integrante faz commits** ao longo da iteração. Quem não tiver nenhuma contribuição registrada no repositório perde também a parcela do grupo.

Cronograma

Quando	O que acontece
12/11 a 15/11	Iteração 0 — a base
16/11 a 17/11	Iteração 1 — H1
18/11 a 19/11	Iteração 2 — H2
19/11	**Trabalho remoto** no horário da aula: o grupo junto, H2 integrada e H3 começada; professor online para dúvidas
20/11 a 22/11	Iteração 3 — H3 e a Refatoração 2
23/11 a 24/11	Iteração 4 — H4
25/11	Fechamento: validação, demo, documento, retrospectiva e branch `entrega-3`
26/11	**Prova 3**, entrega no Teams e defesa individual
02/12	**Prova substitutiva**

> 15/11 e 20/11 são feriados. O cronograma já conta com eles: são dias bons para a iteração que cada um tem na mão, não para combinar o grupo.

O ciclo de cada iteração

As iterações 1 a 4 seguem **os mesmos sete passos**. É isso que faz o trabalho ser incremental: a cada história, o produto roda, está testado e foi visto por outra pessoa.

Passo	O que fazer	Vem da
1. Cortar	Quebrar a história em **fatias** que entram na `main` sozinhas, na ordem em que entregam valor. A primeira é o caminho principal.	Aula 10
2. Decidir	Responder às **decisões do grupo** listadas na história (o que o produto faz no caso de borda). Decisão que muda o modelo vai em `## Decisões de projeto` de `docs/projeto.md`.	Aulas 6 e 11
3. Testar primeiro	Cada critério de aceite vira um `it` com o **nome da regra**, o Dado preparado pela API e o Então que confere o **valor**. Roda **vermelho** antes de existir o código. Mais **um caso limite ou de erro** por história.	Aula 11
4. Implementar	Regra em `src/doacoes.js`, SQL em `src/repositorio.js`, rota em `src/app.js`, tela em `public/index.html`. Vazamento de tipo do banco se conserta **abaixo do contrato**.	Aulas 10 e 12
5. Revisar e integrar	PR com a história, o critério de origem, as fatias e o `## Uso de IA`; revisão de **outro integrante**; branch atualizada; merge; CI da `main` verde.	Aulas 10 e 12
6. Mostrar	Um **passo novo no roteiro da demo** (`docs/demo.md`), com ação exata e resultado esperado — no navegador do celular.	Aula 13
7. Validar e olhar para trás	A linha da história no **quadro de validação** (`docs/validacao.md`) e **uma frase** no rascunho da retrospectiva: o que esta iteração mudou na próxima.	Aula 13

O passo 7 é o que faz o trabalho ser iterativo, não sequencial: se a iteração 2 mostrou que o corte da 1 estava errado, a 3 começa diferente — e a retrospectiva registra isso.

Iteração 0 — A base (até 15/11)

Antes da primeira história nova, o chão tem de estar firme.

1. **O produto roda sobre PostgreSQL.** A migração da Aula 12 está na `main`: `src/db.js` com `pg` mantendo o contrato, `src/repositorio.js` com `$1, $2…`, o `services: postgres` descomentado no CI, `src/doacoes.js` fora do diff da migração. Em `docs/refatoracoes.md`, a **Refatoração 1** com o número de testes verdes antes e depois e o link do CI.
2. **O README está certo.** `## Como rodar` descreve o produto **sobre PostgreSQL** (como subir o banco, a `DATABASE_URL` com um exemplo — nunca a senha real —, os três comandos) e `## Definição de pronto` tem 4 itens ou mais, cada um com onde se confere.
3. **Os estados da doação estão decididos.** Em `## Decisões de projeto` de `docs/projeto.md`, uma linha nova: os estados (`disponivel`, `aceita`, `coletada`), as transições permitidas (as setas do desenho acima) e o que acontece com uma transição que não está no desenho. É o contrato das quatro histórias — sem ele, cada dono inventa a sua regra.
4. **Cada história tem dono e revisor**, escritos no README ou no quadro do grupo.

Pronto quando: a `main` está verde **sobre PostgreSQL**, a Refatoração 1 está registrada, o README descreve como rodar o produto atual, e os estados e as transições estão em `## Decisões de projeto`.

Iteração 1 — H1 · O doador acompanha as suas doações (até 17/11)

> **Como** doador, **quero** ver as doações que publiquei e a situação de cada uma, **para** saber se alguém vai buscar.

Hoje a doação não sabe quem a publicou. Esta história acrescenta o **doador** à doação — e, com isso, a primeira mudança de **schema** depois da migração, sobre uma tabela que já tem linhas.

Critérios de aceite

- **CA1.** Dado que a "Padaria Pão Bom" publicou duas doações e a "Casa Esperança" aceitou uma delas, quando a padaria consulta as suas doações, então vê as duas: uma *disponível* e outra *aceita por Casa Esperança*.
- **CA2.** Dado que outro doador publicou uma doação, quando a "Padaria Pão Bom" consulta as suas, então essa doação não aparece.
- **CA3.** Dado que o doador não é informado na publicação, quando a doação é enviada, então ela é recusada com uma mensagem clara e nada é gravado.

Decisões do grupo

- **As doações já publicadas não têm doador.** O que fazer com elas: um doador padrão, ficarem de fora da consulta, ou outra saída? Escrevam a decisão e o porquê — num piloto de verdade, essas doações seriam de alguém.
- Como o doador se identifica, sem login: pelo nome digitado, por um código, guardado no navegador? A escolha mais simples que o caso aceita é válida — e a limitação dela vai para `## Limitações do produto`.

Fatias sugeridas: (1) gravar o doador na publicação; (2) a consulta das doações de um doador, com a situação; (3) a tela "Minhas doações". Depois de H2, a situação *coletada* aparece aqui sem esforço — se o corte foi bom.

Pronto quando: CA1 a CA3 têm teste, a decisão sobre as doações antigas está registrada, e um doador vê no celular as próprias doações e a situação de cada uma.

Iteração 2 — H2 · O voluntário confirma a coleta (até 19/11)

> **Como** voluntário, **quero** confirmar que busquei a doação, **para** que a ONG e o doador saibam que ela chegou.

É a história que mede o objetivo de impacto da análise: **o tempo entre "disponível" e "coletada"** só existe se a coleta tiver um horário.

Critérios de aceite

- **CA1.** Dado que a "Casa Esperança" aceitou uma doação, quando o voluntário "Rui" confirma a coleta, então ela passa a *coletada*, com o nome do voluntário e o horário da coleta.
- **CA2.** Dado que a doação ainda está *disponível*, quando alguém tenta confirmar a coleta, então a confirmação é recusada com mensagem e a doação não muda.

- **CA3.** Dado que a doação foi coletada, quando o doador consulta as suas doações (H1), então ela aparece como *coletada*.

Decisões do grupo

- Confirmar **duas vezes** a mesma coleta: recusa, ou ignora em silêncio? O teste diz o que o grupo decidiu.
- O nome do voluntário é obrigatório? É dado pessoal: o que vocês guardam, e quem vê?

A inversão de regra. Nesta iteração, **invertam de propósito** a regra do CA2 (deixem confirmar uma doação *disponível*), rodem a suíte, contem os vermelhos e desfaçam. Registrem no PR, na seção `## Inversão de regra`: a regra invertida, os testes que ficaram vermelhos e "inversão desfeita". Se nenhum ficou vermelho, o teste foi reforçado e a inversão repetida.

Pronto quando: CA1 a CA3 têm teste, a inversão está registrada, e a demo mostra uma doação indo de *aceita* para *coletada* no celular.

Iteração 3 — H3 · A ONG devolve uma doação aceita (até 22/11)

> **Como** ONG, **quero** devolver uma doação que aceitei e não vou conseguir buscar, **para** que outra ONG possa aproveitá-la a tempo.

É a única transição que **volta** no desenho dos estados — e a que mais cuidado pede: uma doação devolvida tem de voltar **inteira** para a lista, sem rastro da ONG anterior.

Critérios de aceite

- **CA1.** Dado que a "Casa Esperança" aceitou uma doação, quando ela a devolve com o motivo "sem voluntário hoje", então a doação volta para a lista de disponíveis, sem ONG, e o motivo fica registrado.
- **CA2.** Dado que a doação já foi *coletada*, quando a ONG tenta devolvê-la, então a devolução é recusada com mensagem.
- **CA3.** Dado que a "Casa Esperança" aceitou a doação, quando **outra** ONG tenta devolvê-la, então a devolução é recusada e a doação continua aceita pela Casa Esperança.

Decisões do grupo

- O motivo é obrigatório? Onde ele fica guardado — e quem o vê depois (o doador? a Marta, em H4)?
- A ONG que devolveu pode aceitar a mesma doação de novo?

A Refatoração 2 nasce aqui. Com H3, três operações — aceitar, confirmar a coleta e devolver — começam do mesmo jeito: buscar a doação, recusar se ela não existe, conferir se a transição é permitida. É a **terceira repetição**: a regra de três da Aula 12 diz que é hora de extrair. A ordem: a **primeira fatia de H3** entra como a terceira cópia, com o teste verde; em seguida, em **PR à parte**, a extração, com a suíte **verde antes e verde depois**; as fatias seguintes de H3 já usam a função extraída. Se o grupo enxergou outro cheiro mais doloroso, vale: o que importa é **o cheiro, a dor na próxima mudança e a evidência**. Registrem em `docs/refatoracoes.md`, na **Refatoração 2**.

A revisão registrada. O PR da Refatoração 2 é o candidato natural à **revisão completa**: um integrante que não o escreveu deixa **3 comentários ou mais** em linha (onde, o quê, por quê, sugestão, se bloqueia), o autor responde a cada um, e a descrição ganha a seção `## O que a revisão mudou`. Ponham o link desse PR no fim da Refatoração 1 ou 2 do `docs/refatoracoes.md`: `- Revisão: <link>`.

Pronto quando: CA1 a CA3 têm teste, a Refatoração 2 está na `main` com verde antes e depois e registrada, e a revisão completa está no PR com a seção `## O que a revisão mudou`.

Iteração 4 — H4 · A Marta vê o resultado do mês (até 24/11)

> **Como** coordenadora, **quero** ver o resultado do mês, **para** mostrar aos apoiadores se a comida está chegando mais rápido.

A história que fecha o ciclo da análise: é aqui que o **objetivo de impacto** da Unidade 1 vira número.

Critérios de aceite

- **CA1.** Dado que, em novembro, 3 doações foram publicadas, 2 aceitas e 1 coletada 5 horas depois de publicada, quando a Marta consulta o resultado de novembro, então vê *3 publicadas*, *2 aceitas*, *1 coletada* e *tempo médio até a coleta: 5 h*.
- **CA2.** Dado um mês sem nenhuma coleta, quando a Marta consulta o resultado, então o tempo médio aparece como *sem coletas no mês* — não como zero, nem como erro.
- **CA3.** Dado uma doação publicada em 31/10 e coletada em 01/11, quando a Marta consulta novembro, então o grupo **decide** em que mês ela conta — e o teste diz qual.

Decisões do grupo

- O mês da doação é o da **publicação** ou o da **coleta**? (É o CA3.) E as devolvidas de H3 entram no resultado?
- O tempo médio vem em horas, em minutos, arredondado como?

Dois cuidados que esta unidade já ensinou. (1) **Datas nos testes:** o teste não pode depender do dia em que roda — controlem as datas (o grupo escolhe como) para que o CA1 passe hoje e daqui a um ano. (2) **Tipos do banco:** `COUNT(*)` e médias de datas voltam do PostgreSQL com tipos que o JavaScript não espera; o conserto vai **abaixo do contrato**, no SQL ou no `db.js` — nunca em `src/doacoes.js` nem no teste.

Pronto quando: CA1 a CA3 têm teste que passa em qualquer dia, e a Marta vê o resultado do mês no celular.

Fechamento (25/11)

Com as quatro histórias na `main`, o grupo fecha a iteração. Boa parte disto já foi escrita, linha a linha, no passo 7 de cada iteração.

1. A evidência de funcionamento

- **Clone limpo:** um integrante que **não** escreveu o README clona o repositório numa pasta nova e sobe o produto e os testes **só com ele**.
- **O roteiro da demo** em `docs/demo.md`: **3 minutos** — o problema em uma frase, **a vida de uma doação** (publicada → aceita → coletada, e uma devolvida), **um caso de erro** e o que falta. Cada passo com ação exata e resultado esperado. Um integrante que não o escreveu o executa sem ajuda; o `## Testado por` registra quem, quando e o que mudou.

2. A validação contra a análise — `docs/validacao.md`

- A tabela com **todos** os critérios de aceite do `docs/analise.md` (Unidade 1), **copiados como estão**, e **depois** os critérios de H1 a H4.
- Em cada linha, o **status** — *atende*, *parcial*, *não atende* ou *não verificável* — e a **evidência** que outra pessoa confere: o nome do teste, o passo do roteiro, o link do CI. "Testado" não é evidência.
- Cada *parcial* e *não atende* em **"Critérios que não atendem — por quê"**; cada *não verificável* ou critério que mudou em **"Mudanças de critério"**.

Um "não atende" bem justificado vale mais que um "atende" sem evidência.

3. O documento de fechamento (até 4 páginas)

Não é um arquivo novo: é o par **`docs/refatoracoes.md`** + **`docs/validacao.md`**, com o teto de 4 páginas para os dois somados.

O fechamento pede	Onde está
As refatorações feitas e por quê	`docs/refatoracoes.md` — Refatoração 1 (iteração 0) e Refatoração 2 (iteração 3)
O resultado da validação	`docs/validacao.md` — a tabela e "Critérios que não atendem — por quê"
As limitações do produto	`docs/validacao.md` — **`## Limitações do produto`**, ao fim: o que o produto não faz ou não aguenta hoje e por quê (a identificação do doador sem login, por exemplo), em 3 a 5 linhas
O Uso de IA	`docs/validacao.md` — **`## Uso de IA`**: o que a IA fez em cada história e o que vocês verificaram, com **pelo menos um erro dela que vocês corrigiram**

4. A Retrospectiva 3 — `docs/retrospectivas/3.md`

A partir de `docs/retrospectivas/modelo.md`, meia página, usando as frases do passo 7:

- **o que decidimos** — 2 a 3 decisões, com o porquê (os estados da doação, o mês do CA3 de H4, o que fazer com as doações antigas...), não a lista de tarefas;
- **o que funcionou** e **o que mudaríamos** — um fato das iterações, nunca "nada";
- **próximos passos**;
- **autoavaliação de contribuição:** 100 pontos entre os integrantes, cada linha com o que a pessoa fez de mais relevante **e o link** (PR ou commit). Ela orienta as perguntas da defesa e não entra no cálculo da nota.

5. A entrega

1. **Conferir o checklist** (abaixo), item a item, olhando só o repositório.
2. **Criar o branch** **`entrega-3`**, que congela o que foi entregue:

```
git checkout main
git pull
git checkout -b entrega-3
git push -u origin entrega-3
```

A correção olha esse branch. Commits nele depois do prazo não são considerados.

3. **Cada integrante entrega no Teams**, na atividade do Trabalho 3:

```
Repositório: <url do repositório do grupo>
Branch: entrega-3
Execução do CI: <url da execução verde no GitHub Actions, no branch entrega-3>
Meus commits: <url da lista de commits filtrada por você>
```

Para a última linha: no GitHub, aba **Commits**, filtro **Author**, selecione seu usuário e copie o endereço.

Pronto quando: o produto sobe de um clone novo, o roteiro foi executado por quem não o escreveu, o quadro tem todos os critérios com evidência, os dois documentos cabem em 4 páginas, a retrospectiva soma 100, e o `entrega-3` está no GitHub com o CI verde.

Estrutura do repositório na entrega

```
README.md                  Definição de pronto + Como rodar (sobre PostgreSQL)
src/, tests/               as quatro histórias + os testes
docs/projeto.md            Decisões de projeto: os estados da doação
docs/refatoracoes.md       Refatorações 1 e 2 + a revisão       ┐ documento de
docs/validacao.md          critérios, limitações e Uso de IA    ┘ fechamento (4 p.)
docs/demo.md               roteiro da demo, com o Testado por
docs/retrospectivas/3.md   a retrospectiva da iteração
.github/workflows/ci.yml   CI rodando sobre PostgreSQL
```

Como é corrigido

Artefato do grupo (2,0) — o checklist:

Item	Onde confiro
Entregue no prazo e no formato solicitado.	branch `entrega-3` + entrega individual no Teams
Código executa; demo/roteiro prova o funcionamento; CI verde.	clone do `entrega-3`, `/api/saude`, `docs/demo.md`, aba *Actions*
As quatro histórias estão integradas por PR revisado, e os testes cobrem os critérios de aceite delas.	PRs de H1 a H4, `tests/` (um teste por critério, um limite por história), `## Inversão de regra`
As 2 refatorações estão registradas e protegidas por testes.	`docs/refatoracoes.md`, o PR da migração (sem `src/doacoes.js` no diff), o PR da Refatoração 2 (verde antes e depois), a revisão registrada
Commits e PRs mostram evolução incremental e participação de todos.	um PR por fatia, donos e revisores das histórias, commits por integrante
Validação contra os critérios de aceite da Análise apresentada.	`docs/validacao.md` × `docs/analise.md`
Retrospectiva e seção "Uso de IA" presentes.	`docs/retrospectivas/3.md`, `## Uso de IA` e `## Limitações do produto` no `docs/validacao.md`

Uma história pela metade conta, se o PR disser o que ficou de fora e o quadro de validação a marcar como *parcial*. Uma história escondida não conta.

Defesa individual (1,0) — em 26/11, conforme cada um termina a prova, 2 a 3 minutos comigo:

O que	Pontos
Demonstrar a sua parte e explicar uma refatoração — o que melhorou e como os testes a protegeram	0,4
Responder uma pergunta sobre **outra** parte do trabalho, que você não produziu	0,3
Fazer uma pequena modificação ao vivo, no documento ou no código	0,3

Levem o **notebook com o repositório clonado e o produto rodando**. As perguntas vêm das histórias: *por que a devolução volta a doação para a lista e não a apaga?*, *o que aconteceria com o CA1 de H4 se o banco devolvesse a contagem como texto?*, *em que mês conta a doação que atravessa a virada?* — e sempre: *como vocês saberiam que isso quebrou?*

A **autoavaliação por pares** orienta as perguntas, mas não entra no cálculo. A defesa é **independente por trabalho**.

Gatilho: quem não tiver nenhuma contribuição registrada no repositório nesta iteração perde também a parcela do grupo.

Nível de IA: colaboradora

A IA pode gerar código, testes e rascunhos e revisar o que vocês escreveram. O que ela fez vai registrado — no `## Uso de IA` de cada PR e no do `docs/validacao.md`.

Ela é boa em produzir e ruim em julgar. O que continua sendo de vocês: as **decisões do grupo** de cada história, o corte das fatias, o esperado de cada caso limite, onde consertar um vazamento do banco e o status de cada critério. Dois cuidados que a Unidade 3 já mostrou: a IA **não olha o resto do repositório** — na iteração 3 ela vai sugerir uma quarta cópia da busca em vez de usar a extração — e gera testes que **passam sem conferir a regra**. **"A IA fez assim" não é justificativa aceita** na defesa.

Perguntas frequentes

Nosso grupo já implementou uma destas histórias. Ela conta se estiver na `main` com os critérios desta página testados. No lugar dela, implementem **uma** destas, com o mesmo ciclo: *"a ONG vê as doações que aceitou"* ou *"o endereço de retirada só aparece para a ONG que aceitou"*. Digam no PR qual história ela substitui.

Podemos fazer as histórias em outra ordem? H1 e H2 podem trocar de lugar — nesse caso, o CA3 de H2 (a coletada aparece para o doador) passa para H1. H3 depende de H2 (a coletada não se devolve) e H4 depende de H2 (sem coleta não há tempo médio).

Não deu tempo de fazer a quarta. Entreguem três inteiras e a quarta **pela metade, declarada**: a primeira fatia na `main`, o resto no quadro como *parcial*. Vale mais que quatro histórias que não rodam.

As atividades das Aulas 10 a 12 já valem nota. Preciso refazer? Não. Elas são a base da iteração 0.

A migração precisa estar na `main`? Sim. O produto entregue roda sobre PostgreSQL, e o CI do `entrega-3` mostra os testes rodando sobre ele.

Onde fica o documento de fechamento? Nos dois arquivos que já existem — `docs/refatoracoes.md` e `docs/validacao.md` —, mais a seção `## Limitações do produto`. Não criem um terceiro arquivo repetindo o que eles dizem.

Usamos outra stack. Valem os mesmos cinco compromissos do ADR da stack: repositório público com CI verde, rota de saúde, testes por um comando, os três comandos no README e banco relacional migrado para PostgreSQL. Os nomes de arquivo mudam; o que se pede, não.